35.1 UI 技术选型

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~115k tokens，输出 ~6k tokens（本节）

OpenClaw 的 Web 控制台（Control UI）是一个内嵌在 Gateway 中的单页应用，为用户提供可视化的管理界面。本节介绍其技术栈选型——Lit Web Components 框架、Vite 构建工具，以及装饰器约束。

35.1.1 Lit Web Components 框架

为什么选择 Lit

OpenClaw 没有选择 React、Vue、Svelte 等主流前端框架，而是采用了 Google 的 **Lit** 库（`lit@^3.3`）。这个选择的核心考量是：

因素	Lit	React/Vue
包体积	~7KB（gzip）	40-100KB+
运行时	浏览器原生 Web Components	虚拟 DOM
依赖链	极少外部依赖	生态庞大
嵌入式部署	适合内嵌到 Gateway 进程	构建产物较大
框架锁定	Web 标准（Custom Elements）	框架特定

对于一个嵌入到后端进程中的管理界面来说，轻量级是首要需求——控制台 UI 的构建产物直接打包到 Gateway 中通过 HTTP 静态文件服务提供，不需要单独的前端服务器。

> **衍生解释 — Web Components**
>
> Web Components 是浏览器原生支持的组件化标准，由三项技术组成：
>
> - **Custom Elements**：定义自定义 HTML 标签（如 `<openclaw-app>`）
> - **Shadow DOM**：组件内部的 DOM 树与外部隔离，CSS 样式互不干扰
> - **HTML Templates**：`<template>` 和 `<slot>` 标签用于声明组件的内容结构

> 与 React 的虚拟 DOM 不同，Web Components 直接使用浏览器的原生 DOM API，没有额外的运行时抽象层。

> **衍生解释 — Lit**
>
> Lit 是 Google 在 Web Components 标准之上开发的轻量级库，前身是 Polymer。它的核心贡献是：
>
> 1. **声明式模板**：使用 JavaScript 标签模板字面量（`html`...`` ）描述 UI
> 2. **响应式属性**：通过 `@state()` / `@property()` 装饰器标记状态，变更时自动触发局部重渲染
> 3. **极小体积**：核心运行时仅约 7KB

OpenClaw 中的 Lit 应用

整个控制台 UI 是一个名为 `<openclaw-app>` 的 Custom Element：

```
// ui/src/ui/app.ts
import { LitElement } from "lit";
import { customElement, state } from "lit/decorators.js";

@customElement("openclaw-app")
export class OpenClawApp extends LitElement {
  @state() connected = false;
  @state() tab: Tab = "chat";
  @state() chatMessages: unknown[] = [];
  // ... 100+ 响应式状态字段

  render() {
    return renderApp(this);  // 委托给独立的渲染模块
  }
}
```

在 `index.html` 中只需一行：

```
<openclaw-app></openclaw-app>
```

依赖清单

控制台 UI 的运行时依赖极其精简：

依赖	用途	大小
`lit`	Web Components 框架	~7KB
`marked`	Markdown → HTML 渲染	~40KB
`dompurify`	HTML 消毒（防 XSS）	~15KB
`@noble/ed25519`	设备认证签名	~5KB

总共只有 4 个运行时依赖，整个 UI 的构建产物体积远小于典型的 React 应用。

35.1.2 Vite 构建工具

构建配置

控制台 UI 使用 Vite 7 作为构建工具：

```
// ui/vite.config.ts
export default defineConfig(() => {
  const envBase = process.env.OPENCLAW_CONTROL_UI_BASE_PATH?.trim();
  const base = envBase ? normalizeBase(envBase) : "./";
  return {
    base,                                          // 可配置的 base path
    publicDir: path.resolve(here, "public"),        // 静态资源目录
    optimizeDeps: {
      include: ["lit/directives/repeat.js"],        // 预构建 Lit 指令
    },
    build: {
      outDir: path.resolve(here, "../dist/control-ui"),  // 输出到主项目的 dist/
      emptyOutDir: true,
      sourcemap: true,
    },
    server: {
      host: true,
      port: 5173,
      strictPort: true,
    },
  };
});
```

关键设计决策：

1. **相对路径 base**：默认 `base: "./"` 使构建产物可以在任意 URL 路径下部署。Gateway 通过 `OPENCLAW_CONTROL_UI_BASE_PATH` 环境变量控制挂载点。
2. **输出到主项目 dist**：`outDir` 指向 `../dist/control-ui`（主项目的 `dist/control-ui/` 目录），这样 Gateway 发布时可以将 UI 资源作为子目录一并分发。
3. **独立的 package.json**：`ui/` 目录有自己的 `package.json`，与主项目的依赖完全隔离。构建命令为 `pnpm ui:build`（在主项目中定义的脚本别名）。

开发模式

开发时使用 `pnpm ui:dev` 启动 Vite 的开发服务器（端口 5173），支持 HMR（Hot Module Replacement）。由于 UI 代码直接 import 了主项目 `src/` 下的一些类型和工具函数（如 `src/routing/session-key.js`），开发服务器通过 Vite 的文件系统解析自动处理跨目录的 import。

35.1.3 Legacy Decorators 约束

装饰器兼容性问题

Lit 使用 TypeScript 装饰器（decorators）来声明 Custom Element 和响应式属性：

```
@customElement("openclaw-app")   // 注册自定义元素
export class OpenClawApp extends LitElement {
  @state() connected = false;     // 内部响应式状态
  @property() title = "";         // 外部传入的属性
}
```

TypeScript 对装饰器有两种实现：

版本	TypeScript 配置	标准
Legacy（实验性）	`experimentalDecorators: true`	TC39 Stage 1
标准	`"experimentalDecorators"` 不设置	TC39 Stage 3 / ES2024

Lit 3.x 的装饰器（如 `@customElement`、`@state`）使用 **Legacy 装饰器**语法。这意味着 UI 项目的 TypeScript 配置必须启用 `experimentalDecorators`。

对项目的影响

由于主项目（OpenClaw 后端）可能使用标准装饰器或不使用装饰器，而 UI 子项目必须使用 Legacy 装饰器，两者的 `tsconfig.json` 需要独立配置。这也是 UI 作为独立子项目（有自己的 `package.json`）

的原因之一——避免 TypeScript 配置冲突。

无 Shadow DOM

`OpenClawApp` 类没有覆盖 `createRenderRoot()` 方法来启用 Shadow DOM——这意味着整个控制台 UI 的样式是全局的（通过 `ui/src/styles.css` 和 `ui/src/styles/` 目录定义），而非组件级隔离的。这是一个实用性选择：管理界面不需要样式隔离的严格封装，而全局样式更便于主题切换和统一调整。

本节小结

1. **Lit Web Components** 是控制台 UI 的核心框架，选择它的原因是体积极小（~7KB）、基于浏览器原生标准、适合嵌入式部署场景。
2. **运行时依赖仅 4 个**：Lit、Marked（Markdown 渲染）、DOMPurify（XSS 防护）、ed25519（设备认证）。
3. **Vite 7** 作为构建工具，输出到主项目的 `dist/control-ui/` ，支持通过环境变量配置 base path。
4. **Legacy 装饰器**是 Lit 3.x 的强制要求，UI 子项目需要独立的 TypeScript 配置。
5. **无 Shadow DOM** 设计使全局样式和主题切换更简单，适合管理界面的需求。